

Using GA and D* Lite to Create a Generalized Maze Solving Agent

Group 9: Elena Orozco, Hussain Ali, Emaad Rahman, Kevin Santhosh, Amir Nazemi

<https://github.com/emaadR980/Maze-Solver>

1. Abstract

Using GA and D* Lite, our agent navigates a partially observable 64 x 64 maze containing rotating fire pits, confusion pads, and teleport pads from scratch, with no prior map knowledge aside from knowing the goal location. Our final design combines D* Lite for dynamic pathfinding with an evolutionary controller, a feedforward neural network whose weights are optimized by a genetic algorithm. Before arriving at this design, we built and validated a working Q-learning + A* system. This report documents both approaches, explains the trade-offs that drove our final decision, and presents full training and evaluation metrics on maze alpha and beta. On maze beta, the final agent achieves a 100% success rate, an average of 2587 turns to solution, and a best solve of 384 steps.

2. Introduction

A major goal of the silent cartographer project was to design and implement an intelligent agent capable of navigating unknown 64x64 maze environments containing walls, death pits, teleports, and confusion pads. The agent was required to operate under partial observability: it could not inspect unexplored regions of the maze and could only update its internal model through feedback returned after each turn. This feedback included the agent's current position, wall collisions, death events, confusion status, teleport events, number of actions executed, and goal completion.

The final system developed for this project uses a hybrid architecture combining D* Lite path planning with an evolved neural network controller. D* Lite provides efficient replanning [1] as the agent discovers new walls, hazards, and teleport behavior, while the neural controller supports local decision making in situations involving uncertainty and dynamic hazards. This combination was selected after earlier experimentation with

Q-learning and A* based variants showed limitations in either reliability, scalability, or adaptability.

3. Environment and Maze Parsing

Environment behavior is implemented in `environment.py`, which defines the action space, the `TurnResult` feedback structure, and the `MazeEnvironment` class used during both training and evaluation. Maze images are loaded through `MazeLoader`, imported from [maze.py](#). Each 1024x1024 PNG image is interpreted as a 64x64 grid in which every cell occupies a 16x16 pixel region with a 2 pixel wall border and a 14x14 pixel navigable interior.

`MazeEnvironment` class maintains the authoritative maze state throughout an episode. This includes the complete set of passable cells, confirmed wall edges, start and goal positions, death pit locations, and the current rotation index, confusion pad positions, and teleport pad mappings. On each call to `step()`, the environment processes the submitted action list sequentially. If the agent reaches the goal or dies before the full action list is consumed, any training actions in the batch are discarded, consistent with the project specifications.

The environment additionally tracks per-episode statistics: total turns, death count, confusion events triggered, cells explored, and goal status, which are used for both training fitness evaluation and the post-evaluation reporting described in the project spec. A CNN based loader (`maze_cnn.py`) was developed as an alternative option for maze environment generation, but is not used in the final training or evaluation pipeline.

4. Hazard Detection

Hazard classification is implemented in [maze.py](#) and operates once at load time, prior to any episode. The classifier follows a two stage pipeline build on OpenCV:

Stage 1 - Template Matching (primary). Each cell interior is compared against a library of pre-cropped 14x14 pixel template images using normalized cross-correlation. The template with the highest correlation score above a confidence threshold is accepted as the label. Templates for each hazard type were created using `make_templates.py`, which allows a user to interactively click on maze cells and save the extracted interiors

Stage 2- Color Analysis (fallback). If the template stage cannot produce a confident classification, the classifier applies a color-based fallback. The cell interior is converted to HSV primarily to isolate foreground pixels from the white background using saturation and values thresholds. After foreground pixels are identified, the classifier compares their average BGR color against stored color prototypes for selected hazard classes. Additional color ratio checks help distinguish visually similar hazards, such as orange/yellow teleport pads, fire hazards, and confusion pads. This stage improves reliability when template scores alone are not decisive.

Fire pit cells rotate 90deg between turns. The four rotation states are precomputed from the initially detected pit positions before training begins. The pivot cell for each fire cluster is defined as the cluster member with the minimum Euclidean distance to the cluster's geometric centroid; this cell remains fixed across all rotation states. The rotation index increments at the start of each turn, so the environment always reflects the correct fire configuration without requiring re-detection.

The output of the pipeline is a complete hazard map assigned to MazeEnvironment before the first episode begins. Critically, this map is held by the environment and not exposed to the agent directly. The agent still discovers hazard locations through interaction during navigation, preserving the partial observability requirement of the project.

5. Agent Architecture

Our final agent is implemented in `maze_agent.py` and comprises four tightly integrated components: AgentMemory, StateEncoder, D* Lite planner, and a neural controller evolved by a genetic algorithm. These components are coupled such that the planner consults memory to build its cost graph, the encoder consults both memory and the planner to produce a feature vector, and the neural controller translates that feature vector into an action decision.

5.1. Agent Policies

The agent's behavior is governed by learned policies encoded in the neural network weights and shaped by the fitness function. When the known planning graph is incomplete, the agent prioritizes unexplored frontiers while still biasing movement towards the goal. Known hazards such as death pits and confusion pads are assigned high

traversal costs, while confirmed useful teleporters can be treated as shortcut edges. The agent also penalizes repeated visits, which helps reduce loops. At each decision step, D* Lite provides the global route recommendation, and the neural controller adjusts action choice using the 43 dimensional encoded state.

5.2. Agent Memory

At the beginning of navigation, the agent is blind aside from knowing where the goal is. It learns the inner workings of the maze as it goes. All structural information is inferred purely from the TurnResult feedback the environment returns after each turn [2]. Navigation is initially driven by the D Lite*, which uses Manhattan distance as its heuristic. As training progresses across generations, the GA accumulates a collective set of discovered death pit locations (shared_pits) that is seeded into every new individual at the start of its episode. Wall knowledge is deliberately not shared. Accumulating wall edges across individuals causes D* Lite to incorrectly block valid paths, so each agent rediscovers walls within its own episode [3].

5.3. D* Lite: Incremental Pathfinding

D* Lite search algorithm works backwards. Cost estimates propagate from the goal outward [4]. Each cell maintains two values: $g(u)$ (the current best cost) and $rhs(u)$ (one-step ahead). When a new obstacle is discovered, only cells adjacent to it are marked inconsistent and reprocessed. This makes replanning after each wall discovery significantly cheaper than rerunning a full A* search from scratch [5].

The heuristic is Manhattan distance: $h(u) = |u_row - goal_row| + |u_col - goal_col|$. This is admissible and consistent on a grid, guaranteeing D* Lite always finds the optimal path given the currently known information.

Two extensions are applied on top of the algorithm. First, discovered teleporter source and destination pairs are injected via `notify_new_teleports()`, letting D* Lite route through teleporters as shortcut edges. Second, during training a small visit count penalty (0.05 per prior visit) nudges the planner away from heavily revisited cells without overriding clearly shorter paths.

5.4. Neural Network Controller

D* Lite handles standard navigation: meanwhile, a neural network takes over specifically when D* Lite's recommended next cell is currently on fire and that fire is expected to clear within $\text{MAX_FIRE_WAIT} = 3$ turns. In all other situations D* Lite's action is followed directly.

Architecture: $43 \rightarrow 64 \rightarrow 32 \rightarrow 5$, fully connected. Hidden layers use ReLU. Output uses softmax over the five actions. Weights initialized with the initialization [6] (scaled by $\sqrt{2 / \text{fan_in}}$).

43 input features:

- Goal direction and normalized Manhattan distance **(3)**
- Per-neighbor: wall, pit, fire presence + visit count. 4 directions x 4 = **(16)**
- Current fire rotation phase **(1)**
- Teleporter benefit per direction, estimated path reduction **(4)**
- Stagnation signals: turns since distance decreased, turns since new cell, last wall hit, current cell visits, open directions, dead end flags **(7)**
- Context: distance from start, confusion status, path length, at start flag, on-teleporter flag **(5)**
- D* Lite hint: one hot encoding of its recommended action. UP/DOWN/LEFT/RIGHT/NONE **(5)**
- Fire wait hint: normalized turns until D* Lite's next cell clears **(1)**

= **43** total

D* Lite hint in the input was an important design choice. The neural network can directly see what the planner recommends and learn specifically when to deviate from it (for example: during fire timing decisions).

6. Training Methodology

6.1. Genetic Algorithm

NN weights are not trained by gradient descent. The GA treats each individual's flat weight vector as its genome and evolves the population over generations, avoiding the need for a differentiable reward signal [7].

Parameter	Value
Population Size	80
Elitism	Top 15% copied unchanged
Selection	Tournament, $k=4$
Crossover	Uniform, probability 0.70
Mutation rate	0.10 per weight
Initial σ	0.20, decays $\times 0.995/\text{gen}$, min 0.08
Random immigrants	2% per generation
Phase switch threshold	12 cumulative solvers
Transition generations	20
Max turns/episode	10,000
Early-stop	300 turns with no new cells discovered

Selection: Tournament selection draws 4 random individuals and selects the highest fitness one as a parent [8]. Performed twice per child. The design was meant to balance selection pressure with diversity better than pure elitism.

Crossover: Uniform crossover. Each weight is independently drawn from either parent with 50/50 probability.

Mutation: Gaussian noise $N(0, \sigma^2)$ added to each weight with probability 0.10. σ decays each generation. If the best fitness fails to improve by 100+ points over 15 consecutive generations during OPTIMIZE phase, σ is reset to $\min(0.20, \sigma \times 2.5)$ to escape local optima.

Random immigrants: 2% of each generation is replaced with fresh random controllers to prevent premature convergence [9].

6.2. Two-Phase Fitness Function

Training uses two sequential fitness objectives:

EXPLORE phase (early generations): Rewards breadth (minimum Manhattan distance reached toward the goal, unique cells visited, exploration efficiency). Deaths and wall hits are penalized. Goal reached awards +120,000, dominating any exploration score.

OPTIMIZE phase (after 12 cumulative solvers): Rewards speed and safety.

$$\text{Fitness} = 500,000 - 50 \times \text{deaths} - 20 \times \text{wall_hits}$$

Unsolved episodes fall back to the EXPLORE score. The transition between the phases is automatic and smooth because the fitness function blends linearly from EXPLORE to OPTIMIZE over 20 generations once triggered. This addresses a known bootstrapping problem in evolutionary maze-solving. A purely greedy objective does not provide a useful gradient signal when no individual in the population can yet solve the maze [10].

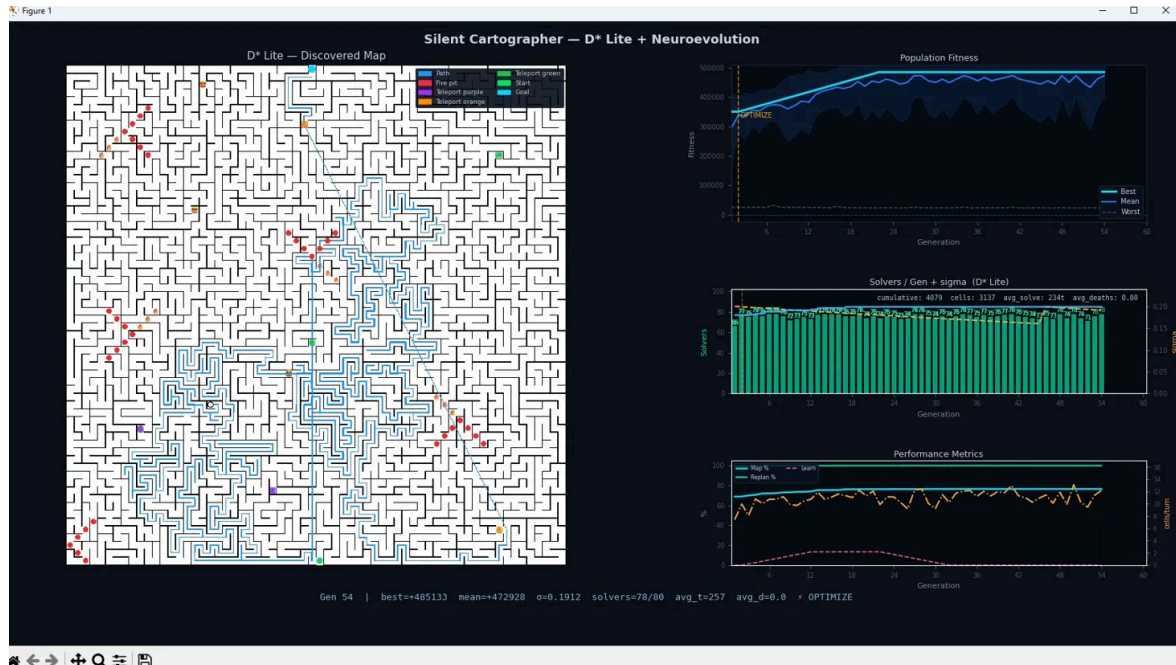


Fig. 1. Visual for training measuring fitness, solvers, maze exploration, sigma, and replanning

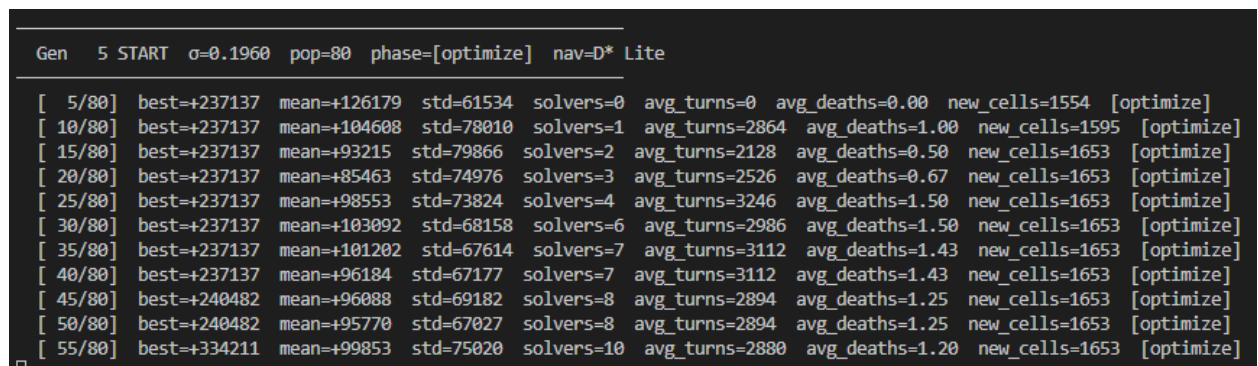


Fig. 2. Bonus metrics print to terminal

7. Evaluation Results

All evaluation results were obtained using the command `[python live_viz.py --test --maze "maze-beta/MAZE_1.png" --turns 10000 --weights best_weights_dstar_v3.npy]`. This mode initializes the environment with the maze beta image, loads the trained weight vector, and runs five episodes with the neural controller operating in greedy mode ($\epsilon = 0$). Beta was never seen during training.

7.1. Maze Metrics (Alpha + Beta + Gamma)

Metric	Alpha	Beta	Gamma
Success Rate	80%	100%	20%
Avg. Path Length (cells)	502	869	578
Avg. Turns to Solution	716	2587	1594
Death Rate	0.20/ep	1.20/ep	0.40/ep

7.2. Bonus Metrics (Alpha + Beta + Gamma)

Metric	Alpha	Beta	Gamma
Exploration Efficiency (cells/turn)	0.2048	0.2138	0.0345
Map Completeness	65.40%	69.50%	43.90%
Replanning Efficiency	100%	60%	100%
Learning Efficiency	18.30%	95.60%	N/A (1 solve)

7.3. Per-Episode Results (Beta Only)

Episode	Result	Steps	Deaths	Cells	Fitness
1	Solved	8711	5	2141	+300454
2	Solved	2865	1	1014	+339212
3	Solved	589	0	435	+350856
4	Solved	384	0	378	+351868
5	Solved	385	0	378	+351880

7.4. Visualization

The figures below present recordings of the agent navigating maze beta during episode 1 and 5 of the evaluation run. These two episodes are selected to illustrate the agent's adaptation: episode 1 represents the agent's first encounter with an entirely unknown layout, while episode 5 reflects performance after four episodes of accumulated map knowledge. The contrast between the two (6579 steps and 4 deaths in episode 1 versus 385 steps and 0 deaths in episode 5) demonstrates that the memory and replanning

architecture produces meaningful improvement across episodes without any weight updates or retraining between runs.

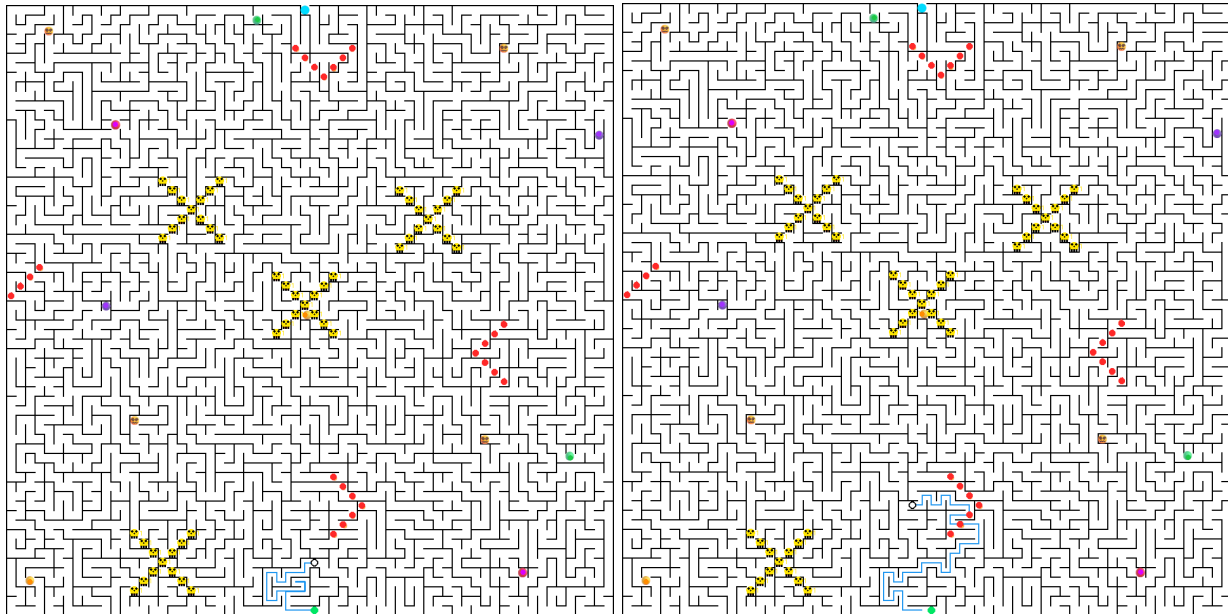


Figure 3. Episode 1, Steps: 6579, Deaths: 4, Cells: 1738 **Figure 4.** Episode 5, Steps: 385, Deaths: 0, Cells: 378

Two visual properties of the figures that require clarification. First, the fire hazards appear in two forms: cells rendered as red squares indicate fire pit locations as detected by the maze parser at load time, while cells highlighted in yellow indicate fire cells that are flagged as lethal by the across episodes. Second, the fire pit clusters may appear to not rotate correctly across frames, this is an artifact of the gif's limited frame rate rather than a failure of the rotation implementation.

8. Comparisons with Earlier Approaches

The project iterated through two prior designs before arriving at the final architecture. The first used a naive BFS with no learning component, serving primarily to validate the environment interface and hazard detection pipeline.

The second replaced BFS with weighted A* and added a Q-learning decision layer, and reliably solved both alpha and beta. However, the approach showed structural limitations at scale. Despite extended training, the saved Q-table covered 2379 discovered states, with 5994 populated state-action entries. Each state encoded position alongside fire

rotation phase, teleport history, and confusion state, meaning the table grew sparsely and many valid states were never visited enough for reliable convergence. Per-step bellman updates also proved noisy under partial observability, since the agent could not distinguish an unlucky outcome from a genuinely suboptimal action. Weighted A* compounded this by replanning from scratch each turn on an incomplete graph, frequently directing the agent into dead ends in unexplored regions.

These limitations motivated the shift to episode level fitness evaluation and D* Lite incremental replanning, which are better suited to the partial observability and long episode budgets of this environment.

9. Conclusion

The agent demonstrates that coupling D* Lite replanning with a neuroevolutionary controller produces a robust navigation system under partial observability. Trained exclusively on maze alpha, the agent achieves a 100% success rate on the unseen maze beta, with a best solve of 385 steps and 0 deaths. The reduction from 6579 steps in episode 1 to 385 steps in episode 5 within a single evaluation run confirms that the memory and replanning architecture supports meaningful adaptation without any weight updates between episodes.

Generalization is bounded by the hazard types present during training. The agent navigates maze layouts reliably when the hazard set is consistent with training, but performance degrades on maze gamma, which introduces arrow pads not encountered during the training run. This suggests the overall approach is effective, while broader generalization will require training or design changes that cover additional hazard types.

10. AI Disclosure

Claude (Anthropic) was used during development for debugging assistance, including generating diagnostic print statements to trace runtime state, for resolving visualization bugs such as fire pit rotation rendering in the live display, and for formatting references to IEEE standards.

11. References

- [1] Y. Chang et al., "D-Lite: Navigation-oriented compression of 3D scene graphs for multi-robot collaboration," *arXiv*, Sep. 13, 2022. [Online]. Available: <https://arxiv.org/abs/2209.06111>
- [2] S. Koenig and M. Likhachev, "D* Lite," in *Proc. AAAI Conf. Artificial Intelligence*, 2002, pp. 476–483.
- [3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.
- [4] S. Koenig and M. Likhachev, "Fast replanning for navigation in unknown terrain," *IEEE Trans. Robot.*, vol. 21, no. 3, pp. 354–363, 2005.
- [5] A. Stentz, "Optimal and efficient path planning for partially-known environments," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 1994, pp. 3310–3317.
- [6] K. He et al., "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015, pp. 1026–1034.
- [7] M. Mitchell, *An Introduction to Genetic Algorithms*. Cambridge, MA, USA: MIT Press, 1998.
- [8] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.
- [9] D. Whitley, "A genetic algorithm tutorial," *Statistics and Computing*, vol. 4, no. 2, pp. 65–85, 1994.
- [10] J. Lehman and K. O. Stanley, "Abandoning objectives: Evolution through the search for novelty alone," *Evolutionary Computation*, vol. 19, no. 2, pp. 189–223, 2011.

12. Beta Per-Episode GIFS

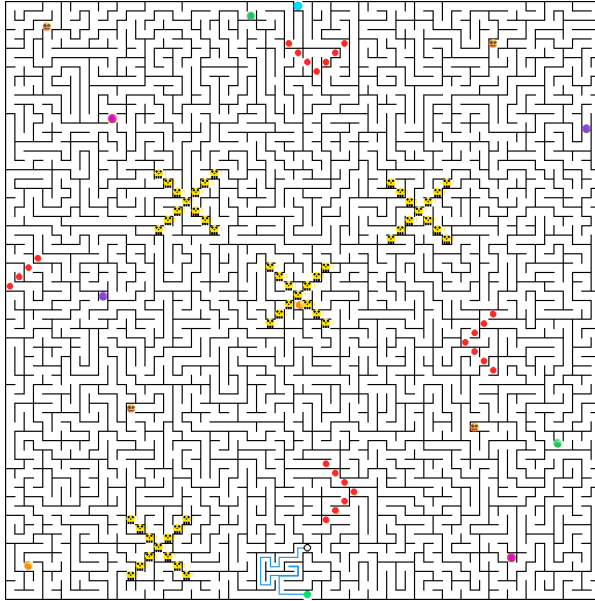


Fig. 5. Episode 1

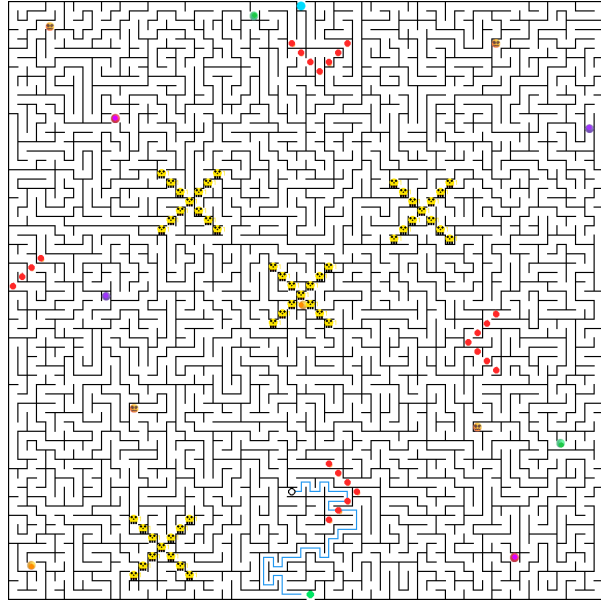


Fig. 6. Episode 2

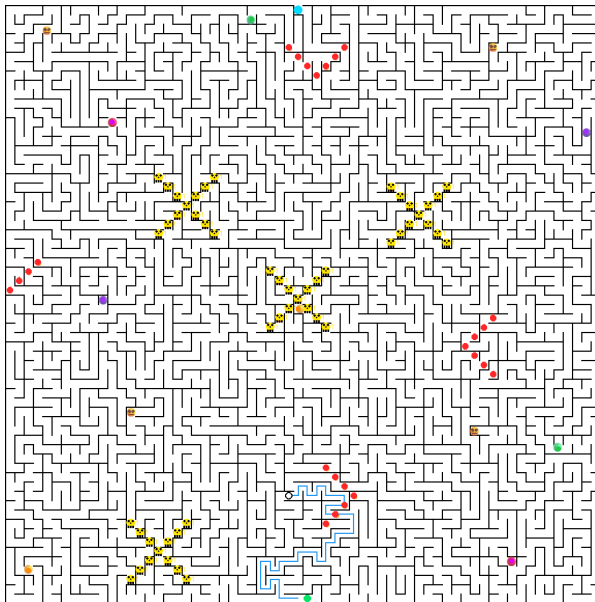


Fig. 7. Episode 3

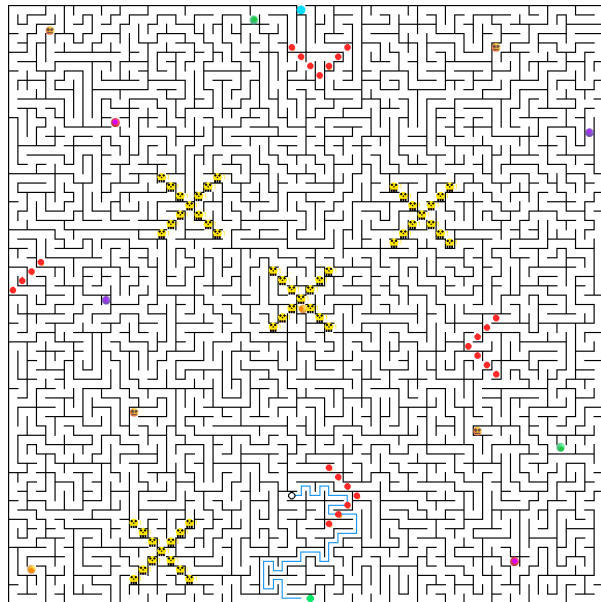


Fig. 8. Episode 4

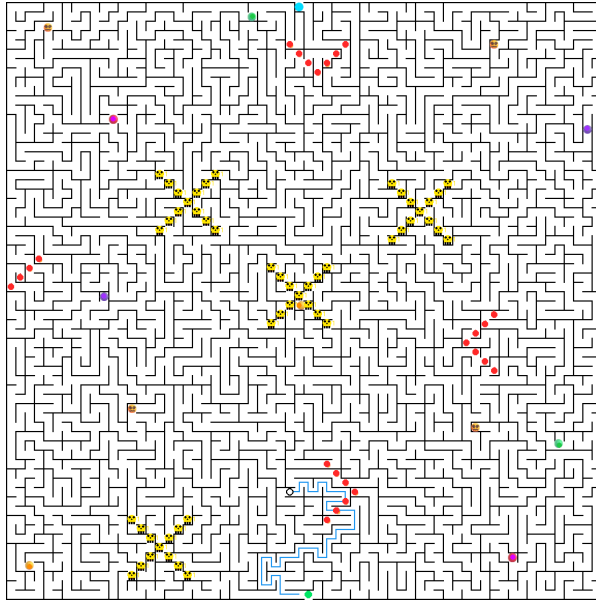
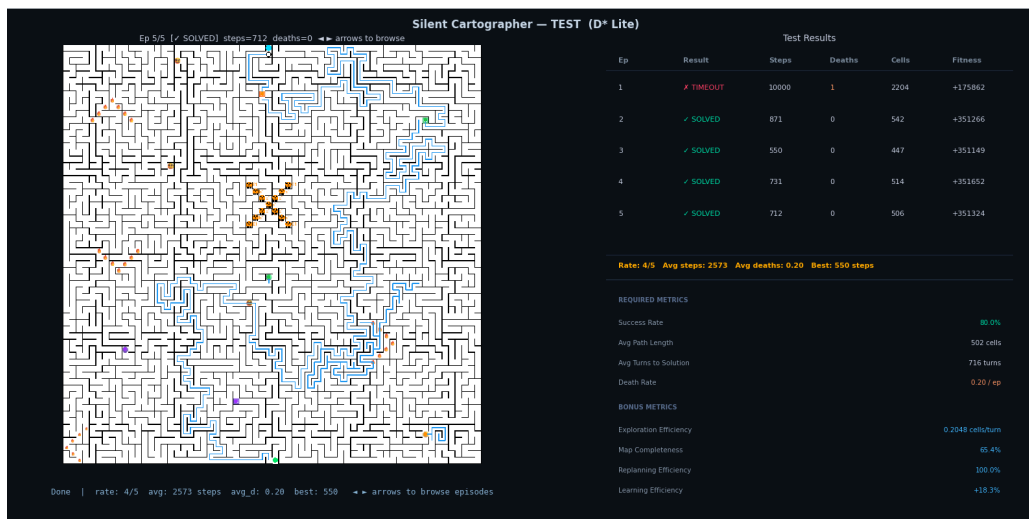


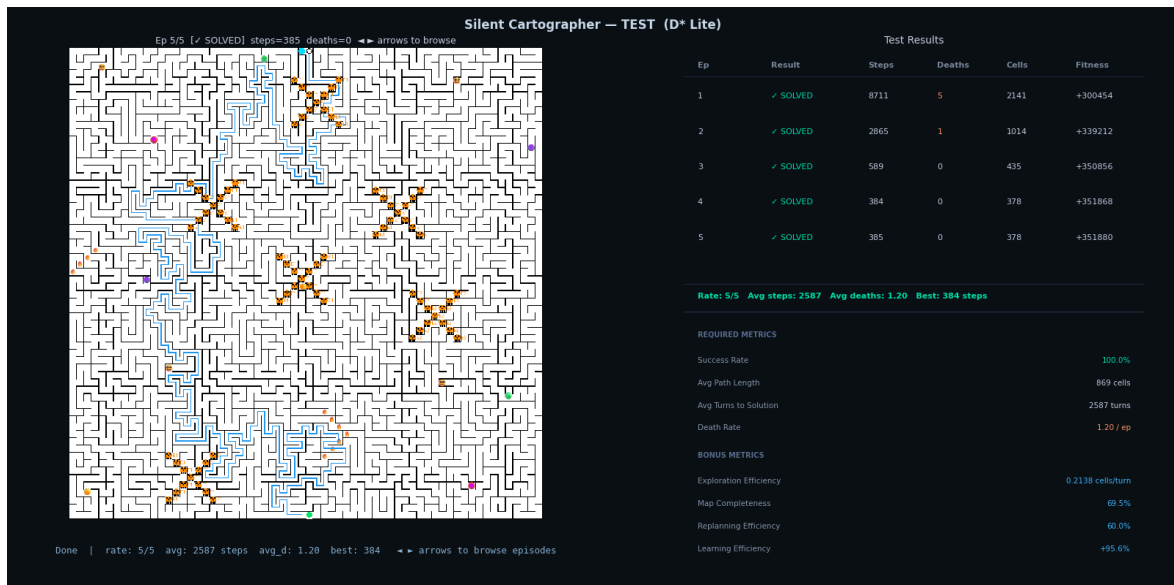
Fig. 9. Episode 5

13. Appendix

13.1. Metrics for Alpha



13.2. Metrics for Beta



13.3. Metrics for Gamma

